Embedded Symplectic Neural Network Inference

Matthew O'Brien

January 12, 2026

1 Overview

This document describes the design, training, fixed-point quantization, and embedded deployment of a Symplectic Neural Network (SympNet) used to model the phase flow of a simple harmonic oscillator (SHO). The system is trained in floating point on a host machine and deployed as a fully fixed-point inference engine on a Raspberry Pi Pico microcontroller.

The objective is to evaluate whether symplectic structure-preserving neural networks retain their theoretical advantages—energy conservation, bounded trajectories, and long-term stability—when executed under strict embedded constraints.

2 Problem Formulation

The simple harmonic oscillator is a Hamiltonian system defined by

$$H(q,p) = \frac{1}{2}p^2 + \frac{1}{2}q^2, \tag{1}$$

with dynamics governed by

$$\dot{q} = \frac{\partial H}{\partial p} = p, \tag{2}$$

$$\dot{p} = -\frac{\partial H}{\partial q} = -q. \tag{3}$$

The true solution traces closed circular orbits in phase space and preserves energy for all time. Any learned model intended for long-horizon rollout must therefore preserve the underlying symplectic geometry.

3 Learned Symplectic Update Used in This Work

This work uses the `method = P` configuration of SympNet, instantiated with two layers and a fixed polynomial degree of two. The model is trained to approximate the discrete-time phase-space flow of a simple harmonic oscillator and is deployed for inference on a microcontroller using fixed-point arithmetic.

This section describes the update rule exactly as implemented, without relying on abstract theoretical formulations.

3.1 Per-Layer Parameterization

Each symplectic layer contains three learned scalar parameters:

- One scalar coefficient a
- Two scalar weights w_q and w_p

No other trainable parameters are present. Each layer operates independently with its own (a, w_q, w_p).

3.2 Polynomial Generator

For a given layer, a scalar polynomial generator is constructed as

$$S(q, p) = a\,(w_q q + w_p p)^2. \tag{4}$$

This polynomial is not learned term-by-term. Instead, its structure is fixed, and learning occurs only through the scalar coefficient a and the linear projection weights w_q and w_p.

3.3 Kick–Drift Update Structure

Each timestep is formed by composing two symplectic substeps: a momentum update (kick) followed by a position update (drift). These correspond to the odd and even layers in the implementation.

3.3.1 Odd Layer: Momentum Kick

In the odd layer, position is held constant and momentum is updated according to

$$q_{n+1} = q_n, \tag{5}$$

$$p_{n+1} = p_n - h\,\frac{\partial S}{\partial q}(q_n, p_n), \tag{6}$$

where h is the timestep.

For the quadratic generator, the derivative is

$$\frac{\partial S}{\partial q} = 2a(w_q q_n + w_p p_n)w_q. \tag{7}$$

This update applies a learned impulse to the momentum state.

3.3.2 Even Layer: Position Drift

In the even layer, momentum is held constant and position is updated according to

$$q_{n+1} = q_n + h\,\frac{\partial S}{\partial p}(q_n, p_n), \tag{8}$$

$$p_{n+1} = p_n. \tag{9}$$

For the quadratic generator, the derivative is

$$\frac{\partial S}{\partial p} = 2a(w_q q_n + w_p p_n)w_p. \tag{10}$$

This update advances the position using the same learned generator.

3.4 One Learned Timestep

In this implementation, a single learned timestep consists of:

1. An odd-layer momentum kick
2. An even-layer position drift

This two-layer composition forms one symplectic update step. Repeated application of this step produces multi-step trajectory rollouts.

3.5 Learning Mechanism

Training data consists of phase-space pairs (x_n, x_{n+1}) generated from a reference simulator. Given an input state $x_n = (q_n, p_n)$, the network predicts the next state $\hat{x}_{n+1}$ by applying the learned kick–drift update.

The loss function penalizes the difference between predicted and true next-step states:

$$\mathcal{L} = \|\hat{x}_{n+1} - x_{n+1}\|^2. \tag{11}$$

Gradients propagate through the explicit polynomial evaluation and symplectic update equations, adjusting the parameters (a, w_q, w_p) in each layer. Each parameter is updated based on how strongly it influences the momentum kick or position drift, effectively tuning the magnitude and direction of the learned phase-space update.

No Hamiltonian is learned explicitly, and no energy-based regularization is used. Stability arises from the symplectic structure of the update itself.

4 Fixed-Point Quantization and Deployment

4.1 Q16.16 Representation

All inference computations are performed using signed Q16.16 fixed-point arithmetic:

- 16 integer bits, 16 fractional bits
- Stored as 32-bit signed integers
- Used consistently for:
  - State variables (q, p)
  - Weights and coefficients
  - Intermediate products and accumulations

This uniform representation eliminates mixed-precision artifacts and simplifies overflow analysis.

4.2 Quantization Pipeline

After training:

1. Learned floating-point parameters are scaled by 2^{16}
2. Values are rounded and stored as 32-bit integers
3. Parameters are embedded as static constants in firmware

No retraining or quantization-aware training is currently required due to the polynomial structure and small parameter count.

5 Embedded Inference State Machine

Inference on the microcontroller is implemented as a deterministic state machine:

1. Host sends initial conditions (q_0, p_0) and requested rollout length
2. MCU iteratively applies symplectic layers for each timestep
3. Predicted states are streamed back to the host

The inference engine supports arbitrary rollout lengths and maintains numerical stability over hundreds of steps.

6 Communication Protocol

A custom USB CDC protocol was implemented to ensure reliable data transfer:

- Explicit packet framing
- Sequence numbers for synchronization
- CRC-8 checksum for corruption detection

Corrupted packets are detected and discarded on the host side, preventing invalid states from contaminating trajectory reconstruction.

7 Validation Methodology

Validation is performed using three trajectory sets:

1. 50 trajectories sampled from the training dataset
2. 15 unseen trajectories within the training bounds
3. 10 unseen trajectories with initial conditions outside the training bounds

Phase-space plots and time-series reconstructions confirm:

- Closed, bounded trajectories
- No long-term drift or energy growth
- Stable behavior under fixed-point arithmetic

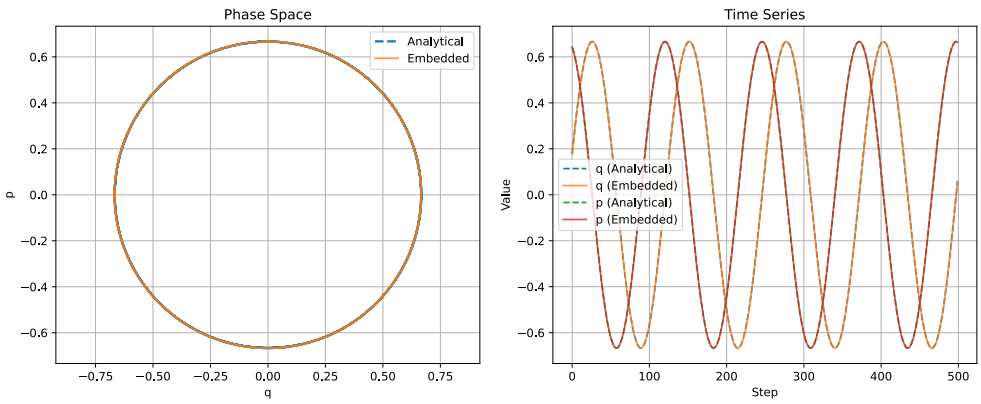

Figure 1: Symplectic phase-space trajectory Embedded Inference Vs Ground Truth

8 Quantitative Results

This section reports accuracy, inference latency, and power consumption metrics measured on the embedded deployment.

8.1 Inference Latency

Inference time was measured for full 500-step trajectory rollouts across training, in-distribution (IID) test, and out-of-distribution (OOD) test sets.

Dataset	Average Inference Time (s)
Training (50 trajectories)	0.237
Test IID (15 trajectories)	0.240
Test OOD (10 trajectories)	0.248

Table 1: Average embedded inference time per 500-step trajectory.

Inference time remains stable across training and test distributions, indicating predictable runtime behavior under deployment conditions.

8.2 Prediction Accuracy

Prediction accuracy is evaluated using Mean Relative Squared Error (MRSE) and average absolute error over full trajectory rollouts.

Dataset	MRSE (%)	Avg. Absolute Error (%)
Training (50 trajectories)	4.69	3.67
Test IID (15 trajectories)	4.65	3.55
Test OOD (10 trajectories)	4.68	8.39

Table 2: Trajectory-level accuracy metrics for embedded SympNet inference.

Accuracy remains consistent between training and IID test trajectories. While absolute error increases for out-of-distribution initial conditions, trajectories remain bounded and stable, reflecting the underlying symplectic structure.

8.3 Power and Energy Consumption

The Raspberry Pi Pico was supplied with a regulated 5.0 V input during inference. Average current draw during active rollout was measured at approximately 0.023 A.

This corresponds to an average power consumption of

$$P = V \cdot I \approx 0.115 \text{ W}. \tag{12}$$

Given an average inference time of approximately 0.24 s per 500-step trajectory, the total energy consumed per trajectory is

$$E_{\text{traj}} \approx 0.028 \text{ J}. \tag{13}$$

This corresponds to an average energy cost of approximately 5.5×10^{-5} J per symplectic update step. These results demonstrate that structure-preserving neural inference can be executed with very low energy overhead on resource-constrained microcontrollers.

8.4 Memory Footprint and Program Size

This section reports the Flash and RAM utilization of the embedded SympNet inference firmware as compiled for the Raspberry Pi Pico (RP2040). All measurements were obtained from the final ELF binary using standard `arm-none-eabi` tooling.

8.4.1 Flash and RAM Utilization

Overall program size was measured using `arm-none-eabi-size`. Flash usage is determined by the combined `text` and `data` sections, while static RAM usage is determined by the combined `data` and `bss` sections.

Metric	Size
Flash usage (code + constants)	25.5 KB
Static RAM usage	11.4 KB
Final UF2 image size	42 KB

Table 3: Embedded firmware memory utilization.

The firmware occupies a small fraction of the RP2040's available 2 MB Flash and 264 KB SRAM, leaving substantial headroom for future extensions.

8.4.2 Symplectic Inference Context Size

Symbol-level analysis was performed using `arm-none-eabi-nm` to identify dominant static memory consumers. The primary RAM allocation corresponds to the symplectic inference context structure, which contains the fixed-point state variables, parameters, and intermediate buffers required for rollout.

$$\texttt{symplecticInferenceContext} \approx 8.0 \text{ KB} \tag{14}$$

This structure accounts for the majority of runtime RAM usage. All inference state is statically allocated at compile time, and no dynamic memory allocation is used during inference.

8.4.3 Memory Determinism

Because all parameters, state variables, and buffers are statically allocated, memory usage is fixed and deterministic across executions. Stack and heap usage do not vary during runtime, and inference execution does not rely on dynamic allocation or recursion.

This design ensures predictable memory behavior and simplifies deployment on resource-constrained microcontrollers.

9 Planned Extensions

With the successful deployment and benchmarking of the symplectic neural network inference pipeline, the focus of this project will now shift toward establishing a conventional neural network baseline for comparison.

9.1 Vanilla Neural Network Baseline

The next phase will implement a standard feedforward neural network trained to predict the phase-space flow of the simple harmonic oscillator. To ensure a fair and meaningful comparison, the initial model will be intentionally constrained to minimal depth and parameter count, beginning with a two-layer architecture and the smallest number of weights possible.

This design choice is motivated by the extreme parameter efficiency of the symplectic model, which achieves stable long-horizon rollouts using only a small number of learned coefficients. The vanilla network will therefore be incrementally scaled only as necessary to match accuracy, stability, or rollout behavior.

9.2 Training and Inference Pipeline

The vanilla network will be trained using the same dataset generation pipeline and evaluated using the same trajectory-based accuracy metrics. Inference will be implemented using explicit numerical operations to enable direct translation to a fixed-point embedded implementation.

If additional capacity is required to achieve competitive performance, model depth and width will be increased incrementally while tracking the resulting impact on accuracy, stability, latency, and energy consumption.

9.3 Embedded Deployment

On the embedded side, the existing infrastructure—including USB communication, packet framing, fixed-point arithmetic, and accuracy evaluation—will be reused without modification. Activation functions will be selected or adapted to ensure compatibility with fixed-point arithmetic and efficient execution on the microcontroller.

This approach allows the comparison to focus exclusively on model structure and learning dynamics rather than differences in deployment or instrumentation.

9.4 Comparative Evaluation

Once both models are deployed, a direct comparison will be performed across:

- Prediction accuracy
- Long-horizon stability
- Inference latency
- Energy per trajectory
- Memory footprint

This comparison will quantify the practical benefits of symplectic structure preservation relative to conventional neural networks under identical embedded constraints.